After you've spawned the thread, you can update the block start, remembering to advance it past that last element ⓳, and store the future for the last value in the current chunk into the vector of futures so it will be picked up next time around the loop ⓴.

Before you process the final chunk, you need to get an iterator for the last element ㉑, which you can pass in to `process_chunk` ㉒. `std::partial_sum` doesn't return a value, so you don't need to do anything once the final chunk has been processed. The operation is complete once all the threads have finished.

OK, now it's time to look at the `process_chunk` function object that actually does all the work ❶. You start by calling `std::partial_sum` for the entire chunk, including the final element ❷, but then you need to know if you're the first chunk or not ❸. If you are *not* the first chunk, then there was a `previous_end_value` from the previous chunk, so you need to wait for that ❹. In order to maximize the parallelism of the algorithm, you then update the last element first ❺, so you can pass the value on to the next chunk (if there is one) ❻. Once you've done that, you can just use `std::for_each` and a simple lambda function ❼ to update all the remaining elements in the range.

If there was *not* a `previous_end_value`, you're the first chunk, so you can just update the `end_value` for the next chunk (again, if there is one—you might be the only chunk) ❽.

Finally, if any of the operations threw an exception, you catch it ❾ and store it in the promise ❿ so it will propagate to the next chunk when it tries to get the previous end value ❹. This will propagate all exceptions into the final chunk, which then just rethrows ⓫, because you know you're running on the main thread.

Because of the synchronization between the threads, this code isn't readily amenable to rewriting with `std::async`. The tasks wait on results made